Data Analysis using Python Libraries

A **Python library** is a collection of pre-written code that we can use in our programs instead of writing everything from scratch.

For example, suppose we want to calculate the average of 1 million numbers.

Without specialized libraries, we would have to write our own code to:

- store the numbers
- iterate through them
- add them
- calculate the average
- perform other mathematical operations

Python libraries provide ready-made functions and data structures for these tasks.

Common Python libraries for data analysis

Library	Main Purpose
NumPy	Numerical computation and arrays
Pandas	Data manipulation and analysis
Matplotlib	Data visualization
Seaborn	Statistical visualization
SciPy	Scientific and mathematical computing
Scikit-learn	Machine learning
Plotly	Interactive visualization

A typical data-analysis workflow can be represented as:

Data → NumPy/Pandas → Analysis → Matplotlib/Seaborn → Visualization → Insights

Importing a Library

For example:

```
import numpy

numbers = numpy.array([10, 20, 30, 40, 50])

print(numbers)
```

Using an Alias

It is common to give a library a shorter name.

NumPy is normally imported as np.

```
import numpy as np
```

Now instead of writing:

numpy.array()

we can write:

np.array()

Example:

```
import numpy as np
marks = np.array([75, 80, 65, 90, 85])
print(marks)
```

What is NumPy?

NumPy stands for **Numerical Python**.

It is one of the most important Python libraries for:

- numerical calculations
- arrays
- mathematical operations
- statistical calculations
- matrix operations
- scientific computing
- data analysis

The most important data structure in NumPy is the **ndarray**, or **N-dimensional array**.

Why Do We Need NumPy?

Consider a Python list:

marks = [70, 80, 90, 60, 75]

Suppose we want to add 5 marks to every student's marks.

With a normal Python list, we cannot simply write:

marks + 5

This gives an error.

We need a loop:

```
marks = [70, 80, 90, 60, 75]
new_marks = []
for mark in marks:
    new_marks.append(mark + 5)
print(new_marks)
```

Output:

[75, 85, 95, 65, 80]

With NumPy:

```
import numpy as np
marks = np.array([70, 80, 90, 60, 75])
new_marks = marks + 5
print(new_marks)
```

Output:

```
[75 85 95 65 80]
```

This is one of the major advantages of NumPy.

NumPy performs operations on entire arrays.

This is called **vectorized computation**.

Creating a NumPy Array

The basic syntax is:

np.array()

Example:

```
import numpy as np

numbers = np.array([10, 20, 30, 40, 50])

print(numbers)
```

Output:

```
[10 20 30 40 50]
```

Python List vs NumPy Array

Python list

marks = [10, 20, 30, 40]

NumPy array

marks = np.array([10, 20, 30, 40])

The NumPy array is specifically designed for numerical operations.

For example:

```
import numpy as np

marks = np.array([10, 20, 30, 40])

print(marks * 2)
```

Output:

[20 40 60 80]

Whereas with a Python list:

marks = [10, 20, 30, 40]

print(marks * 2)

Output:

[10, 20, 30, 40, 10, 20, 30, 40]

This is because Python interprets * 2 on a list as **repetition**, whereas NumPy interprets it as **element-wise multiplication**.

One dimensional array

A one-dimensional array is similar to a list.

Python

```python
import numpy as np

numbers = np.array([10, 20, 30, 40, 50])

print(numbers)
```

We can access individual elements using indexes.

Python

```python
print(numbers[0])
print(numbers[2])
print(numbers[4])
```

Output:

```
10
30
50
```

10. Negative Indexing

NumPy also supports negative indexes.

Python

```python
import numpy as np

numbers = np.array([10, 20, 30, 40, 50])

print(numbers[-1])
print(numbers[-2])
```

Output:

```
50
40
```

`-1` means the last element.

Two-Dimensional NumPy Array

A two-dimensional array can be visualized as a table.

Python

```python
import numpy as np

marks = np.array([
    [80, 75, 90],
    [70, 85, 88],
    [95, 90, 92]
])

print(marks)
```

Output:

```
[[80 75 90]
 [70 85 88]
 [95 90 92]]
```

Accessing Elements in a 2D Array

The syntax is:

array[row, column]

Example:

print(marks[0, 0])

Since indexing starts from 0:

marks[0,0] = 80

marks[0,1] = 75

marks[0,2] = 90

marks[1,0] = 70

marks[1,1] = 85

marks[1,2] = 88

Rows and Columns

Consider:

```
marks = np.array([
    [80, 75, 90],
    [70, 85, 88],
    [95, 90, 92]
])
```

First row

```
print(marks[0])
```

Output:

```
[80 75 90]
```

Third column

```
print(marks[:, 2])
```

The shape property tells us the dimensions of an array.

```
import numpy as np

marks = np.array([
    [80, 75, 90],
    [70, 85, 88],
    [95, 90, 92]
])

print(marks.shape)
```

Output:

```
(3, 3)
```

Another example:

numbers = np.array([10, 20, 30, 40, 50])

print(numbers.shape)

Output:

(5,)

There are 5 elements in a one-dimensional array.

Number of Dimensions — ndim

The ndim property tells us the number of dimensions.

```
numbers = np.array([10, 20, 30, 40])

print(numbers.ndim)
```

Output:

1

For a two-dimensional array:

```
marks = np.array([
    [80, 90],
    [70, 85]
])

print(marks.ndim)
```

Output:

2

Number of Elements — size

size tells us the total number of elements.

```
marks = np.array([
    [80, 90],
    [70, 85],
    [95, 88]
])

print(marks.size)
```

Output:

6

Creating Arrays with arange()

np.arange() creates a sequence of numbers.

Syntax:

np.arange(start, stop, step)

Example:

```
import numpy as np

numbers = np.arange(1, 11)

print(numbers)
```

Output:

```
[ 1  2  3  4  5  6  7  8  9 10]
```

Notice that **11 is not included**.

Example with step

```
numbers = np.arange(0, 20, 2)


print(numbers)
```

Output:

```
[ 0  2  4  6  8 10 12 14 16 18]
```

This is useful for generating sequences.

19. Creating Arrays with `zeros()`

Python

```python
zeros = np.zeros(5)

print(zeros)
```

Output:

```
[0. 0. 0. 0. 0.]
```

Creating a 3 × 4 array:

Python

```python
zeros = np.zeros((3, 4))

print(zeros)
```

Output:

```
[[0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]]
```

20. Creating Arrays with `ones()`

Python

```python
ones = np.ones(5)

print(ones)
```

Output:

```
[1. 1. 1. 1. 1.]
```

Two-dimensional:

Python

```python
ones = np.ones((2, 3))

print(ones)
```

Output:

```
[[1. 1. 1.]
 [1. 1. 1.]]
```

21. Creating an Array Filled with a Specific Value

Use `np.full()`.

Python

```
numbers = np.full(5, 10)

print(numbers)
```

Output:

```
[10 10 10 10 10]
```

For a 3 × 3 array:

Python

```
numbers = np.full((3, 3), 7)

print(numbers)
```

Output:

```
[[7 7 7]
 [7 7 7]
 [7 7 7]]
```

22. Identity Matrix

An identity matrix has 1s on the main diagonal and 0s elsewhere.

Python

```python
identity = np.eye(3)

print(identity)
```

Output:

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

This is useful in matrix calculations.

ARRAY OPERATIONS

NumPy allows arithmetic operations directly on arrays.

```
import numpy as np
a = np.array([10, 20, 30])
b = np.array([2, 4, 5])
```

Addition

```
print(a + b)
```

Output:

```
[12 24 35]
```

Subtraction

```
print(a - b)
```

Output:

```
[ 8 16 25]
```

Multiplication

```
print(a * b)
```

Output:

```
[ 20  80 150]
```

Division

```
print(a / b)
```

Output:

```
[5. 5. 6.]
```

Scalar Operations

We can perform an operation between an entire array and a single number.

marks = np.array([60, 70, 80, 90])

print(marks + 5)

Output:

[65 75 85 95]

Similarly:

print(marks - 5)

[55 65 75 85]

print(marks * 2)

[120 140 160 180]

print(marks / 10)

[6. 7. 8. 9.]

This is called **broadcasting**.

Suppose monthly sales are:

```
sales = np.array([25000, 30000, 28000, 35000, 40000])
```

Suppose sales increase by 10%.

```
new_sales = sales * 1.10

print(new_sales)
```

Output:

```
[27500. 33000. 30800. 38500. 44000.]
```

Suppose temperatures in Celsius are:

celsius = np.array([0, 10, 20, 30, 40])

Formula:

Fahrenheit = Celsius × 9/5 + 32

Python:

fahrenheit = celsius * 9/5 + 32

print(fahrenheit)

Output:

[32. 50. 68. 86. 104.]

Mathematical Functions

NumPy provides many mathematical functions.

Square root

```
numbers = np.array([4, 9, 16, 25])

print(np.sqrt(numbers))
```

Output:

```
[2. 3. 4. 5.]
```

Power

```
numbers = np.array([2, 3, 4])

print(np.power(numbers, 2))
```

Output:

```
[ 4  9 16]
```

Absolute value

```
numbers = np.array([-10, -5, 0, 5, 10])

print(np.abs(numbers))
```

Output:

```
[10  5  0  5 10]
```

NUMPY STATISTICAL FUNCTIONS

30. Sum

Python

```
marks = np.array([70, 80, 90, 60, 75])

print(np.sum(marks))
```

Output:

```
375
```

---

31. Mean

Mean means average.

Python

```
marks = np.array([70, 80, 90, 60, 75])

print(np.mean(marks))
```

Output:

```
75.0
```

Median

```
marks = np.array([70, 80, 90, 60, 75])


print(np.median(marks))
```

Output:

75.0

The median is the middle value after sorting.

Sorted data:

60, 70, 75, 80, 90

Middle value = 75.

33. Minimum and Maximum

Python

```python
marks = np.array([70, 80, 90, 60, 75])

print(np.min(marks))
print(np.max(marks))
```

Output:

```
60
90
```

---

34. Standard Deviation

Standard deviation tells us how spread out the values are.

Python

```python
marks = np.array([70, 80, 90, 60, 75])

print(np.std(marks))
```

Variance

```
marks = np.array([70, 80, 90, 60, 75])


print(np.var(marks))
```

Variance measures the squared deviation from the mean.

What is Percentile in Statistics?

A percentile is a statistical measure that tells us the value below which a certain percentage of observations in a dataset falls.

Consider the following marks arranged in ascending order:
$10, 20, 30, 40, 50, 60, 70, 80, 90$
The median is 50, which is the 50th percentile under the usual median interpretation.

Percentile	Interpretation
25th percentile (P25)	About 25% of observations are at or below this point
50th percentile (P50)	Median; middle of the distribution
75th percentile (P75)	About 75% of observations are at or below this point
90th percentile (P90)	About 90% of observations are at or below this point

For example, if the 95th percentile of a website's response time is 2 seconds, approximately 95% of measured requests completed in 2 seconds or less.

NumPy calculates the position using:

$$\text{Position} = \frac{75}{100}(n-1)$$

Percentiles

NumPy can calculate percentiles.

```
marks = np.array([50, 60, 70, 80, 90, 100])

print(np.percentile(marks, 50))
```

Output:

75.0

```
import numpy as np
marks = [10, 20, 30, 40, 50, 60, 70, 80, 90]
print(np.percentile(marks, 75))
```

Output: 70

- The 75th percentile is 70 marks for this dataset using NumPy's default method.
- It indicates that 70 is the value at the 75th-percentile point. In this example, 6 out of the 9 observations are strictly below 70, and 7 are at or below 70.

Comparing NumPy Arrays

Suppose:

```
marks = np.array([45, 67, 72, 81, 55])
```

We can ask:

```
print(marks > 60)
```

Output:

```
[False  True  True  True False]
```

This gives a **Boolean array**.

Filtering Values

We can use that condition to extract values.

```
marks = np.array([45, 67, 72, 81, 55])

print(marks[marks > 60])
```

Output:

```
[67 72 81]
```

Suppose we want marks between 50 and 80.

```
marks = np.array([35, 45, 55, 65, 75, 85, 95])

result = marks[(marks >= 50) & (marks <= 80)]

print(result)
```

Important

For NumPy arrays, use:

&

for AND.

Use:

|

for OR.

where() Function

np.where() is very useful for applying conditions.

Example:

```
marks = np.array([35, 45, 67, 82, 91])

result = np.where(marks >= 40, "Pass", "Fail")

print(result)
```

Output:

```
['Fail' 'Pass' 'Pass' 'Pass' 'Pass']
```

Array Slicing

Slicing allows us to select part of an array.

```
numbers = np.array([10, 20, 30, 40, 50, 60])


print(numbers[1:4])
```

Output:

```
[20 30 40]
```

Other Slicing Examples

numbers = np.array([10, 20, 30, 40, 50, 60])

First three:

print(numbers[:3])

Output:

[10 20 30]

From index 3 onwards:

print(numbers[3:])

Output:

[40 50 60]

Every second element:

print(numbers[::2])

Output:

[10 30 50]

Reverse the array:

print(numbers[::-1])

Output:

[60 50 40 30 20 10]

Row-wise and Column-wise Operations

Consider:

```
marks = np.array([
    [80, 70, 90],
    [60, 75, 85],
    [90, 95, 88]
])
```

Calculate total of all marks:

```
print(np.sum(marks))
```

Output:

733

Sum of Each Row

print(np.sum(marks, axis=1))

Output:

[240 220 273]

Meaning of axis=1

axis=1 means perform the operation **across columns**, producing one result for each row.

So:

Student 1: 80 + 70 + 90 = 240

Student 2: 60 + 75 + 85 = 220

Student 3: 90 + 95 + 88 = 273

Sum of Each Column

print(np.sum(marks, axis=0))

Output:

[230 240 263]

Meaning of axis=0

Perform the operation **down the rows**, producing one result for each column.

48. Mean of Each Student

Python

```
print(np.mean(marks, axis=1))
```

Output:

```
[80.         73.33333333 91.        ]
```

This gives the average mark of each student.

---

49. Mean of Each Subject

Python

```
print(np.mean(marks, axis=0))
```

Output:

```
[76.66666667 80.         87.66666667]
```

This gives the average mark in each subject.

50. Transpose

Transpose changes rows into columns.

Python

```
print(marks.T)
```

Original:

```
[[80 70 90]
 [60 75 85]
 [90 95 88]]
```

Transpose:

```
[[80 60 90]
 [70 75 95]
 [90 85 88]]
```

This is written as:

Python

```
marks.T
```

Flattening an Array

Suppose:

```
matrix = np.array([
    [1, 2, 3],
    [4, 5, 6]
])
```

We can convert it into a one-dimensional array:

```
print(matrix.flatten())
```

Output:

```
[1 2 3 4 5 6]
```

NumPy is also used for matrix calculations.

60. Matrix Addition

Python

```python
A = np.array([
    [1, 2],
    [3, 4]
])

B = np.array([
    [5, 6],
    [7, 8]
])

print(A + B)
```

Output:

```
[[ 6  8]
 [10 12]]
```

61. Matrix Multiplication

There is an important distinction between:

Python

```
A * B
```

and:

Python

```
A @ B
```

`*` performs **element-wise multiplication**.

Python

```
print(A * B)
```

Output:

```
[[ 5 12]
 [21 32]]
```

`@` performs **matrix multiplication**.

Python

```
print(A @ B)
```

A COMPLETE NUMPY DATA-ANALYSIS EXAMPLE

This is a good program to demonstrate after teaching the individual concepts.

Student Marks Analysis

```
import numpy as np

marks = np.array([45, 67, 82, 56, 91, 73, 38, 88, 64, 79])
print("Marks:", marks)
print("Number of students:", marks.size)
print("Total marks:", np.sum(marks))
print("Average marks:", np.mean(marks))
print("Highest marks:", np.max(marks))
print("Lowest marks:", np.min(marks))
print("Median marks:", np.median(marks))
print("Standard deviation:", np.std(marks))
print("Passed students:", marks[marks >= 40])
print("Distinction students:", marks[marks >= 75])
```

Suppose a company records monthly sales for five products.

```
import numpy as np

sales = np.array([
    [50000, 55000, 60000],
    [45000, 48000, 52000],
    [70000, 75000, 80000],
    [30000, 35000, 40000],
    [65000, 68000, 72000]
])
```

Rows represent products and columns represent three months.

Total sales print(np.sum(sales))

Sales per product print(np.sum(sales, axis=1))

Sales per month print(np.sum(sales, axis=0))

Average sales per product print(np.mean(sales, axis=1))

Highest sales print(np.max(sales))

Products with any sales above ₹75,000

```
print(sales[sales > 75000])
```

Thank You..